

암호이용활성화

HTML5 암호기술 이용 안내서

2015. 12.



미래창조과학부



한국인터넷진흥원

제 · 개정 이력

순번	제 · 개정일	제 · 개정 내역
1	2015.12	"HTML5 암호기술 이용 안내서" 발간
2		
3		
4		

CONTENTS



제1장 개요	06
제 1절 배경 및 목적	06
제 2절 활용 대상 및 구성	07
제2장 HTML5 개요	10
제 1절 웹의 발전	10
제 2절 HTML5 기술 개요 및 특성	11
제3장 WebCrypto API 소개 및 사용 방법	16
제 1절 WebCrypto API 개요	16
제 2절 WebCrypto API의 활용 대상 및 구성	16
제 3절 WebCrypto API의 개념 및 범위	17
제 4절 WebCrypto API 제공 암호 알고리즘	18
제 5절 WebCrypto API 사용 방법	22
제4장 HTML5기반 국산 블록암호 알고리즘 사용 방법	30
제 1절 국산 블록암호 알고리즘 소개(SEED, HIGHT)	30
제 2절 암호 알고리즘 운영모드 및 패딩 방법	30
1. 운영모드	30
2. 패딩 방법	33
제 3절 HTML5 기반 국산 블록암호 알고리즘 사용 방법	36
1. SEED	36
2. HIGHT	47
제5장 구현 유의사항	60
제 1절 기밀 정보의 저장	60
제 2절 웹서비스 보호	61
[부록 1] 용어 정리	64
[부록 2] 파일 정보	68
[부록 3] 참고 자료	69





제1장 개요

제1절 배경 및 목적

제2절 활용 대상 및 구성

01

제1장 개요

제1절. 배경 및 목적

웹(Web)은 1989년 스위스 유럽원자핵공동연구소(CERN)의 연구자들이 HTML(HyperText Markup Language), URL(Unified Resource Locator), HTTP(HyperText Transfer Protocol)이라는 세 가지 기술을 기초로 이기종 환경(운영체제, 개발환경)에서도 데이터를 쉽게 공유하기 위한 목적으로 개발되었다.

이후 1994년 기술 표준화를 위한 W3C(World Wide Web Consortium)가 창립되면서 웹 기술은 눈부신 발전을 이루었으며, 현재는 연구데이터 뿐만 아니라 뉴스, 공공서비스, 멀티미디어, 전자금융 등 실 생활에 밀접한 다양한 서비스를 제공하고 있다.

국내에는 2000년 초반 초고속 인터넷 인프라 확산 정책이 진행되면서 다양한 웹 서비스가 급속도로 발전되었다. 특히, 공인인증서 등의 관련 인프라를 구축하면서 전자결제, 인터넷 뱅킹 등 웹 기반의 서비스 또한 빠른 속도로 확산되었다. 이러한 서비스는 당시 웹 자체의 기술만으로는 필요한 기능을 충분히 구현하지 못하고, 웹 브라우저를 확장하는 각종 비표준 기술을 사용하게 되었다. 이러한 비표준 기술은 HTML5 표준 발표 지연과 보안, 인증, 멀티미디어 등의 기능에 대한 이용자 요구를 충족시키기 위해 더욱 빠르게 확산되었다.

하지만, 각 브라우저 별로 개발된 비표준 기술에 의해 웹 브라우저별 호환성에 제약이 발생하게 되었으며, 이로 인해 웹 서비스 구축에 많은 어려움을 겪게 되었다. 이러한 어려움에 대처하기 위해 한 번의 개발로 다양한 웹 브라우저 및 디바이스에 웹 서비스를 구축하기 위해 기능 및 인터페이스를 일원화하고 이를 HTML5 표준에 포함하려는 노력이 계속되고 있다.

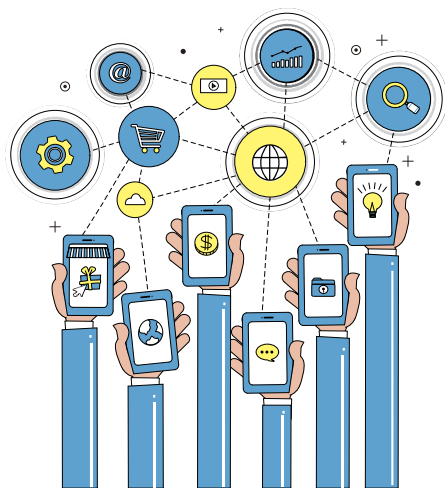
현재 HTML5 표준에는 멀티미디어, 데이터처리, 보안, 인증 등에 대한 내용이 포함되어 있으며, 이를 통해 웹 브라우저의 기본 기능만으로 해당 기능을 사용하는 것을 목적으로 하고 있다.

이에, 본 안내서에서는 WebCrypto API를 기반으로 하여 HTML5 표준 내의 보안, 인증 등에 사용되는 암호기술 및 활용 방법 등을 소개한다.

제2절 활용 대상 및 구성

본 안내서는 HTML5 웹표준 암호기술 이용에 관한 내용을 소개한다. 민감한 정보를 관리하는 기업 및 공공기관 등의 시스템 관리자 및 보안 담당자, 개발자가 HTML5 암호기술을 쉽게 이해하고, 이를 활용하여 암호 기능을 구현하는데 ‘HTML5 암호기술 이용 안내서’를 활용할 수 있다. 따라서 웹 어플리케이션 설계, 개발, 이용에 관심이 있는 누구나 활용할 수 있다.

본 안내서에서는 WebCrypto API 소개 및 사용 방법, HTML5 기반 국산 블록암호 알고리즘 사용 방법, 구현 시 보안 고려사항 등에 대해 소개한다.





암호이용활성화 HTML5 암호기술 이용 안내서



제2장 HTML5 개요

제1절 웹의 발전

제2절 HTML5 기술 개요 및 특성

02

제2장 HTML5 개요

제1절. 웹의 발전

1989년 시작된 웹은 현재는 실생활에 밀접한 서비스를 제공해주고 있으며, 때문에 이제는 없어서는 안 될 기술로 발전해왔다. 이러한 웹 기술의 진화 발전 과정은 일반적으로 [표 2-1]과 같이 4단계로 구분된다.

☑ [표 2-1] 웹 기술의 진화 발전 과정

단계	기간	특징
웹 문서의 시대	1989년 ~ 1999년	HTML, URL, HTTP 기술에 기초한 데이터 공유가 목적
웹 서비스의 시대	2000년 ~ 2004년	다양한 디바이스, 서비스, 멀티미디어 연결이 목적
웹 플랫폼(웹 2.0) 시대	2005년 ~ 2009년	다양한 서비스가 운영 가능한 플랫폼 기반 수립
웹 앱, 모바일, N-Screen 시대	2010년 ~ 현재	통합 응용 플랫폼으로서의 웹

웹 기술은 장치 독립적인 표현 언어를 사용하여 정보를 표현함으로써 표준 브라우저만으로 정보를 교환할 수 있다는 장점을 가지고 있다. 웹 기술은 1993년 HTML 1.0, 1997년 HTML 4.0과 1999년 HTML4.01규격이 만들어지며, 폭발적으로 성장하였다.

확장성이 부족한 HTML의 단점을 보완하고자 W3C는 1999년부터 XML을 기반으로 하는 XHTML(eXtensible Hypertext Markup Language) 1.0을 시작으로 2009년 XHTML 2.0 개발을 추진하였다.

하지만, HTML은 단순한 장점이 있지만, 확장이 어려웠고, XHTML은 확장성이 좋은 장점이 있지만, 지나치게 복잡하였다. 이러한 이유로 XHTML의 표준화는 계속 지연되었고, 다양한 기술적 진화 내역을 포함한 새로운 마크업 언어의 필요성이 대두되었다. 이에 업계 전문가들은 2004년 WHATWG (Web Hypertext Application Technology Working Group)을 구성하고 보다 단순하면서도 확장성을 가지며 이를 통해 다양한 웹 어플리케이션에 효과적으로 사용가능한 HTML 5.0규격 작업이 시작되었다. W3C는 2008년 새로운 HTML규격을 만들기 위한 HTML WG을 구성하고 WHATWG규격을 기초로 한 새로운 HTML5 표준안을 만들기 시작하였다.

HTML5 표준은 기존 HTML 표준으로는 불가능했던 다양한 처리를 별도의 추가적인 확장 기능 없이 가능하게 하는 것에 있다. 이를 위해 HTML5에서는 웹 어플리케이션 개발에 도움을 줄 수 있는 다양한 기능과 API를 제공하고 있으며, 구글/애플/마이크로소프트/모질라 등을 비롯한 많은 웹 브라우저 개발사는 기능 개선과 함께 자사의 규격을 표준에 반영하기 위해 노력 중이다.

제2절. HTML5 기술 개요 및 특성

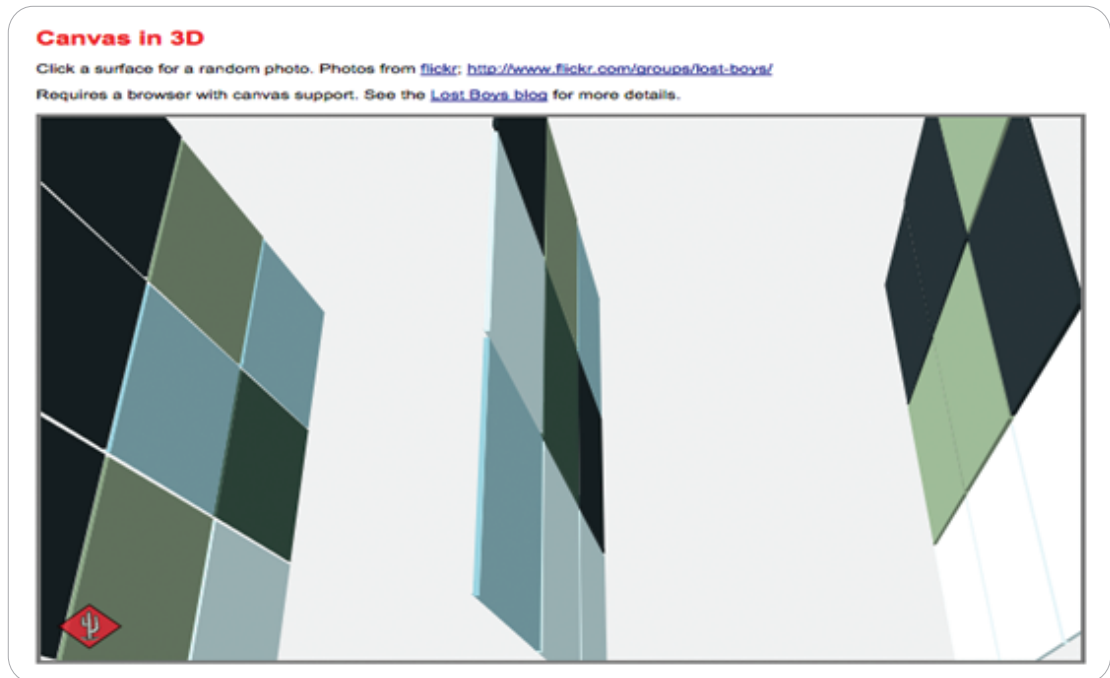
HTML5는 기존 HTML 4.1과 XHTML표준의 단점을 보완하고, HTML 표준만으로 불가능했던 다양한 처리를 추가적인 확장 기능 없이 사용 가능하도록 하는 것을 목적으로 하고 있다. 이를 위해 HTML5에는 [표 2-2]와 같은 웹 어플리케이션 개발에 필요한 다양한 기능을 제공하고 있다.

☑ [표 2-2] HTML5의 주요 기술 및 특징

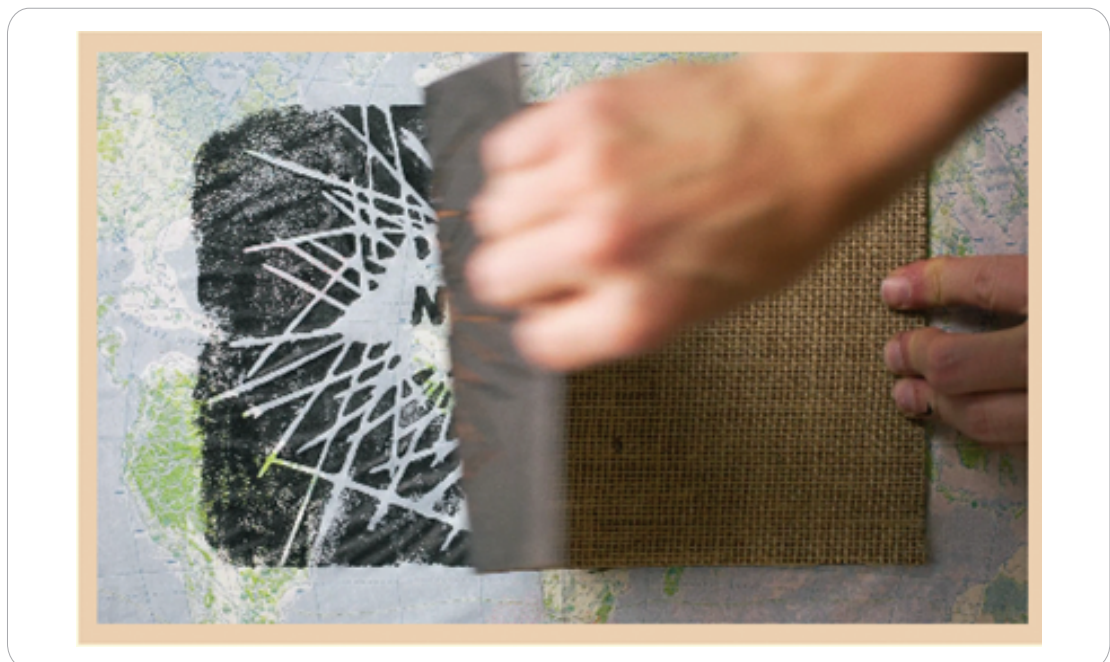
기술 분류	주요 특징
Web Storage	웹 사이트 정보를 사용자 PC 저장 방식 제공, 기존 쿠키 용량 제한 및 트래픽 관련 단점 해소
IndexedDB	웹 브라우저에서 많은 량의 구조화된 데이터를 인덱스를 이용해서 높은 성능으로 검색할 수 있는 기능 제공
Web Socket	웹 상에서 양방향 통신이 가능한 API 제공
File API	로컬 파일을 접근할 수 있는 API 제공
Application Cache	파일을 브라우저에 캐시하고 이를 통해 오프라인 처리, 속도 향상, 서버 부하 감소 기능을 제공
Drag & Drop	웹 페이지로 Drag & Drop 가능한 API 제공
Web Worker	비 동기 백그라운드 처리 API 제공
Web Form	사용자 입력 양식이 개선
2D Drawing	웹 상에서 2차원 그래픽 표현 태그(Canvas), API 지원
SVG	XML기반의 2차원 벡터 그래픽을 표현하는 외부 마크업 사용 가능
3D Drawing	Canvas 3D(WebGL)을 통한 3D 그래픽 지원 WebGL기반의 3D 어플리케이션, 게임 개발용 3D 자바스크립트 엔진
Multimedia	비디오, 오디오 지원 기능의 자체 지원을 통해 강력한 멀티미디어 기능 제공
Push	서버로부터 데이터를 푸시 받을 수 있는 Server-Sent Events 제공
Semantics	보다 구조화되고 다양한 기능의 HTML 태그 제공
Device Access	GPS, 카메라, 동작 센서 등 디바이스의 하드웨어 기능을 웹에서 직접 제어할 수 있는 기능
Connectivity	클라이언트와 서버 간의 효율적인 통신 기능 제공을 통한 웹 기반 커뮤니케이션 효율 강화
CSS3 Styling Effect	기존 웹 문서의 변경과 성능 저하 없이 웹 어플리케이션의 UI (스타일, 효과 등) 기능 강화

(그림 2-1, 2-2, 2-3)은 HTML5 표준의 다양한 기술을 사용한 서비스의 예시이다.

☑ [그림 2-1) Canvas 3D를 사용한 화면 효과



☑ [그림 2-2) 실시간 동영상 스트리밍



✓ [그림 2-3) 캔버스/ CSS를 사용한 예제



이와 같이 HTML5는 기존에 HTML만으로 처리할 수 없었던 많은 서비스를 별도의 외부 확장 없이 웹 브라우저만으로 제공할 수 있는 많은 기능을 제공해준다.

다음 장에서는 이러한 HTML5 표준 중 웹 어플리케이션에서 암호관련 기능을 제공하는 WebCrypto API(Web Cryptography API)에 대해서 기술한다.





제3장 WebCrypto API 소개 및 사용 방법

제1절 WebCrypto API 개요

제2절 WebCrypto API의 활용 대상 및 구성

제3절 WebCrypto API의 개념 및 범위

제4절 WebCrypto API 제공 암호 알고리즘

제5절 WebCrypto API 사용 방법

제3장 WebCrypto API 소개 및 사용 방법

WebCrypto API에 대해서는 WebCrypto API 개요, WebCrypto API의 활용 대상 및 구성, WebCrypto API의 개념 및 범위, WebCrypto API 제공 암호 알고리즘, WebCrypto API 사용 방법으로 나누어 설명한다.

제1절 WebCrypto API 개요

W3C는 암호 관련 기술을 표준화하여 기존 비표준 방식으로 개발된 기능을 웹 어플리케이션에서 별도의 설치 없이 처리하는 것을 목표로 2011년 11월에 Web Cryptography Working Group을 만들고 관련 표준화 작업을 진행해왔으며, 2014년 12월에 최종 권고 후보안을 만들었다.

WebCrypto API(Web Cryptography API)는 실제 키에 대한 접근 요청을 제외한 전자서명 생성 및 검증, 해싱, 암호화, 복호화, 키 생성, 키 유도, 키 불러오기, 키 내보내기, 키 탐색 등을 수행할 수 있는 인터페이스의 집합으로 구성되어 있다.

제2절 WebCrypto API의 활용 대상 및 구성

WebCrypto API는 사용자 에이전트(브라우저)를 통해 실제 키를 노출하지 않고 암호화/복호화 등을 처리할 수 있는 저 레벨 인터페이스를 정의한다. 해당 인터페이스를 통해 암호화/복호화를 이용한 다양한 웹 어플리케이션을 개발할 수 있으며, 활용할 수 있는 분야는 아래와 같다.

– 다중 요소 인증

: 아이디/비밀번호 방식의 사용자 인증 방식을 암호에 기반을 둔 방식으로 대체할 수 있다. 인증서를 사용한 방식은 물론 TLS 클라이언트 인증서와의 연동 등으로 보다 나은 인증 방법을 제공할 수 있다.

– 보호된 문서 교환

: 비밀키로 문서를 암호화하고, 해당 비밀키를 인증된 사용자의 공개키로 암호화 하는 방법을 사용하여 안전하게 문서를 교환할 수 있다.

– 클라우드 저장소

: 원격 서비스 저장소(클라우드)에 사용자의 데이터를 저장할 때 해당 데이터의 암호화를 통해 안전하게 보호받기를 원한다. 이때 서비스 제공자는 사용자가 선택한 개인키 또는 비밀키를 WebCrypto API를 사용하여 업로드 할 수 있다.

– 문서 서명

: 웹 어플리케이션은 실제 서명이 아닌 전자서명을 통해서 업무를 처리할 수 있으며, 이 경우 별도로 생성된 키로 WebCrypto API를 사용하여 문서에 대한 전자서명을 처리할 수 있다.

– 데이터 무결성 보호

: 로컬에 저장된 데이터가 오프라인 공격으로 변경되지 않기를 바란다. 이 경우 서버는 서버의 개인키로 로컬에 저장된 데이터에 대해서 전자서명을 수행하고, 클라이언트는 해당 데이터를 서버의 공개키를 사용하여 검증하여 데이터의 무결성을 확인할 수 있다.

– 자바스크립트(Javascript) 객체 서명 및 암호화 (JOSE)

: IETF의 JOSE 워킹 그룹에서 정의된 메시지 구조와 포맷을 통해 서명과 암호화를 처리할 수 있으며, JOSE 키 형식(JWK)으로 인코딩된 키(공개키, 비밀키)를 사용할 수 있다.

현재 국내에서 암호 관련 기술을 사용하고 있는 분야로는 공인인증서, 인터넷 뱅킹, 전자 결제, 통신 채널 암호화 등이 있다. HTML5의 WebCrypto API는 해당 서비스가 제공하는 모든 기술을 표준 기술로 대체하기에는 아직 부족한 부분이 있지만, 해당 분야의 웹 어플리케이션이 가능한 부분에 대해서 비표준 기술의 사용을 최소화하는 방향으로 나아가는데 큰 도움이 될 것으로 보인다.

제3절 WebCrypto API의 개념 및 범위

WebCrypto API는 암호 관련 기능을 별도의 확장 기능이 아닌 표준으로 처리하는 것을 목적으로 만들어졌다. 하지만, 서비스가 아니라 표준이기 때문에 특정 업체의 환경이나 기술을 강요하거나 권장하지 않는다. 아래는 WebCrypto API 표준의 개념 및 표준이다.

– 추상화

: WebCrypto API는 다양한 플랫폼과 표준화된 암호 API 사이의 공통 기능과 특성에 초점을 맞추고 특정 업체의 구현에 따른 기능 또는 특성에 관련된 부분을 최대한 배제한다. 이러한 추상화는 실제 키 저장소에 대한 구현을 개발자가 고민하지 않으면서도 키 생성, 키 관리, 키 교환을 가능하게 해주며, WebCrypto API는 실제 키를 접근하는 대신 필요한 기능을 제공해주는 API를 제공하는 것을 목적으로 한다.

– 암호화 알고리즘

: 암호 방식의 실제 구현은 사용자 에이전트 별로 다를 것이고, 또한 환경(정부 또는 산업 규제, 지적 재산권 문제, 운영 환경 문제, 로컬 정책 등)에 따라 다를 수 있기 때문에 규격에서는 알고리즘 독립적으로 사용될 수 있는 공통집합, 공통 프레임워크 및 알고리즘 별 적합성 요구 사항 집합에 대해서 정의한다.

– 운영

: 웹 API는 암호 공급자 또는 모듈의 개념을 노출하지 않지만, 각 키는 내부적으로 암호 공급자 또는 모듈과 연동되어 필요한 암호 동작을 정확히 수행할 것을 보증한다.

– 기저 암호의 구현

: 웹 어플리케이션을 개발하는 사용자는 암호작업을 위해 별도의 개발을 진행하는 것이 아니라, HTML5 표준을 만족하는 사용자 에이전트가 제공하는 기능을 제공하는 것으로 추정한다. 이를 위해서 CryptoKey 객체를 제공하고 이 객체는 [[handle]]라는 내부 슬롯의 사용을 통해 자바스크립트 실행환경과 암호 기반 라이브러리와 연결을 제공한다.

– 키 저장소

: WebCrypto API는 저장소 메커니즘에 대해 명시하지 않고, CryptoKey객체를 사용하여 접근하도록 하였다. 이를 통해 현존하고나 미래에 생겨날 웹 저장소 메커니즘이 CryptoKey객체를 저장할 수 있다면 WebCrypto API를 통해 필요한 기능을 제공받을 수 있도록 하였다.

– WebCrypto API가 관여하지 않는 범위

: WebCrypto API는 웹 어플리케이션이 키를 생성, 검색, 조작할 수 있도록 제공하지만, 스마트 카드와 같은 특정 유형의 키 저장소에 대해서 권한 설정에 대해서 정의하지 않는다.

제4절 WebCrypto API 제공 암호 알고리즘

WebCrypto API는 저 레벨의 암호 요소 및 알고리즘을 사용하기 위한 메소드 셋트인 SubtleCrypto 인터페이스, 키 객체를 표현하기 위한 CryptoKey 인터페이스와 SubtleCrypto를 포함한 범용 암호화 기능 인터페이스인 Crypto 인터페이스 등으로 구성된다.

– CryptoKey 인터페이스

CryptoKey 인터페이스는 사용자 에이전트에 의해 관리되는 키에 대한 정보를 표현하고, 일관된 인터페이스를 제공하기 위해 존재한다. CryptoKey는 [표 3-1]과 같이 정의되어 있다.

☑ [표 3-1] CryptoKey 인터페이스

```
[IDL]
enum KeyType { "public", "private", "secret" };
enum KeyUsage { "encrypt", "decrypt", "sign", "verify", "deriveKey", "deriveBits", "wrapKey",
"unwrapKey" };

[Exposed=(Window,Worker)]
interface CryptoKey {
    readonly attribute KeyType type;
    readonly attribute boolean extractable;
    readonly attribute object algorithm;
    readonly attribute object usages;
};
```

– SubtleCrypto 인터페이스

SubtleCrypto 인터페이스는 WebCrypto API를 통해 사용할 수 있는 저 레벨의 암호 요소 및 알고리즘에 대한 메소드 셋트로 [표 3-2]과 같이 정의 되어 있으며, WebCrypto API를 통해 기능을 수행하기 위한 핵심적인 인터페이스이다.

☑ [표 3-2] SubtleCrypto 인터페이스

```
[IDL]
enum KeyFormat { "raw", "spki", "pkcs8", "jwk" };
[Exposed=(Window,Worker)]
interface SubtleCrypto {
    Promise<any> encrypt(AlgorithmIdentifier algorithm,
                        CryptoKey key,
                        BufferSource data);
    Promise<any> decrypt(AlgorithmIdentifier algorithm,
                        CryptoKey key,
                        BufferSource data);
```

```

Promise<any> sign(AlgorithmIdentifier algorithm,
                  CryptoKey key,
                  BufferSource data);

Promise<any> verify(AlgorithmIdentifier algorithm,
                   CryptoKey key,
                   BufferSource signature,
                   BufferSource data);

Promise<any> digest(AlgorithmIdentifier algorithm,
                   BufferSource data);

Promise<any> generateKey(AlgorithmIdentifier algorithm,
                        boolean extractable,
                        sequence<KeyUsage> keyUsages );

Promise<any> deriveKey(AlgorithmIdentifier algorithm,
                      CryptoKey baseKey,
                      AlgorithmIdentifier derivedKeyType,
                      boolean extractable,
                      sequence<KeyUsage> keyUsages );

Promise<any> deriveBits(AlgorithmIdentifier algorithm,
                       CryptoKey baseKey,
                       unsigned long length);

Promise<any> importKey(KeyFormat format,
                      (BufferSource or JsonWebKey) keyData,
                      AlgorithmIdentifier algorithm,
                      boolean extractable,
                      sequence<KeyUsage> keyUsages );

Promise<any> exportKey(KeyFormat format, CryptoKey key);

Promise<any> wrapKey(KeyFormat format,
                    CryptoKey key,
                    CryptoKey wrappingKey,
                    AlgorithmIdentifier wrapAlgorithm);

Promise<any> unwrapKey(KeyFormat format,

```

```

    BufferSource wrappedKey,
    CryptoKey unwrappingKey,
    AlgorithmIdentifier unwrapAlgorithm,
    AlgorithmIdentifier unwrappedKeyAlgorithm,
    boolean extractable,
    sequence<KeyUsage> keyUsages );
};

```

[표 3-2]에 기술된 바와 같이 WebCrypto API는 암호화(encrypt), 복호화(decrypt), 서명(sign), 검증(verify), 다이제스트(digest), 키 생성(generateKey), 키 유도(deriveKey), 비트 유도(deriveBits), 키 가져오기(importKey), 키 내보내기(exportKey), 키 싸기(wrapKey), 키 풀기(unwrapKey)의 기능을 제공한다.

각 기능에 따라 사용 가능한 알고리즘은 [표 3-3]과 같다.

☑ [표 3-3] 기능별 사용 가능 알고리즘

기능	사용 가능 알고리즘
암호화	RSA-OEAEP, AES-CTR, AES-CBC, AES-GCM, AES-CFB
복호화	RSA-OEAEP, AES-CTR, AES-CBC, AES-GCM, AES-CFB
서명	RSASSA-PKCS1-v1_5, RSA-PSS, ECDSA, AES-CMAC, HMAC
검증	RSASSA-PKCS1-v1_5, RSA-PSS, ECDSA, AES-CMAC, HMAC
다이제스트(해시)	SHA-1, SHA-256, SHA-384, SHA-512
키 생성	RSASSA-PKCS1-v1_5, RSA-PSS, RSA-OEAEP, ECDSA, ECDH, AES-CTR, AES-CBC, AES-CMAC, AES-CFB, AES-KW, HMAC, DH, PBKDF2
키 유도	ECDH, DH, CONCAT, HKDF-CTR, PBKDF2
비트 유도	ECDH, DH, CONCAT, HKDF-CTR, PBKDF2
키 가져오기	RSASSA-PKCS1-v1_5, RSA-PSS, RSA-OEAEP, ECDSA, ECDH, AES-CTR, AES-CBC, AES-CMAC, AES-CFB, AES-KW, HMAC, DH
키 싸기	RSA-OEAP, AES-CTR, AES-CBC, AES-GCM, AES-CFB, AES-KW
키 풀기	RSA-OEAP, AES-CTR, AES-CBC, AES-GCM, AES-CFB, AES-KW

또한, 보안을 위해 추천하는 알고리즘은 아래와 같다.

- HMAC using SHA-1
- HMAC using SHA-256
- RSASSA-PKCS1-v1_5 using SHA-1
- RSA-PSS using SHA-256 and MGF1 with SHA-256
- RSA-OAEP using SHA-256 and MGF1 with SHA-256
- ECDSA using P-256 curve and SHA-256
- AES-CBC

충돌저항성(안전성)이 80비트 보안강도 이하를 제공하는 SHA-1 해시 알고리즘 및 SHA-1을 이용하는 암호기술 사용을 권장하지 않음

제5절 WebCrypto API 사용 방법

WebCrypto API는 사용하고자 하는 기능에 대해 알고리즘과 필요한 인자를 JSON 문법에 따라 설정하여 해당 기능을 수행하게 된다. WebCrypto API를 사용하는 방법은 지원하는 알고리즘 별로 필요한 인자를 설정하는 것 이외에는 거의 동일한 방식을 따른다. 본 문서에서는 대칭키를 이용하여 암호화를 진행하는 방법과 비 대칭키를 이용하여 전자서명을 하는 방법에 대한 예제를 기술하였다.

(그림 3-1)은 대칭키 암호인 AES의 CBC 운영모드인 AES-CBC 알고리즘을 사용하여 암호화 하는 예제이다.

☑ [그림 3-1] AES-CBC를 사용한 암호화 예제

```
//Plain Text를 위한 버퍼 설정
var clearDataArrayBufferView
= convertStringToArrayBufferView("AES Encryption");

// 키 생성 알고리즘(AES-CBC)과 인자 설정
var aesAlgorithmKeyGen = {
  name: "AES-CBC", //사용 알고리즘 설정
  length: 128 // AesKeyGenParams (키 길이 설정)
};

// 암호화 알고리즘(AES-CBC)과 인자 설정
var aesAlgorithmEncrypt = {
  name: "AES-CBC", // 사용 알고리즘 설정
  iv: window.crypto.getRandomValues(new Uint8Array(16)) //iv 설정
};

// 암호화에 사용할 키 생성하고,
// 생성에 성공하면 해당 키를 사용하여 대칭키(AES-CBC)를 사용하여 암호화 한다.
window.crypto.subtle.generateKey(aesAlgorithmKeyGen, false, ["encrypt"]).then(
function(aesKey) {
  var cipher = window.crypto.subtle.encrypt(aesAlgorithmEncrypt, aesKey, clearDataArrayBufferView);
  return cipher;
}
).then (
function(cipher)
{
  console.log("The ciphertext is:" + ab2Hex(cipher) );
}
);
```

(그림 3-2)는 (그림 3-1)의 예제를 사용하여 암호화를 수행한 화면이다.

☑ (그림 3-2) AES-CBC 수행 화면

대칭키 (AES) 키 생성 - 암호화

AES 암호화 로그

Plain Text : AES Encryption

AES Key

- type:secret
- algorithm:AES-CBC
- length:128

The ciphertext is: 2bcc50e2e4f8b55bba317582b2a158

(그림 3-3)은 해시 알고리즘(SHA-256)을 사용하여 다이제스트를 생성하는 예제이다.

☑ (그림 3-3) SHA-256를 사용한 다이제스트 생성 예제

```
window.crypto.subtle.digest(  
  {  
    name: "SHA-256",  
  },  
  new Uint8Array([1,2,3,4]) //해싱을 수행할 데이터  
)  
.then  
(function(hash){  
  console.log(ab2Hex(hash));  
})  
.catch(function(err){  
  console.error(err);  
});
```

(그림 3-4)는 (그림 3-3)의 예제를 사용하여 해싱을 수행한 화면이다.

☑ (그림 3-4) SHA-256 해싱 수행 화면

해시(SHA-256)

SHA-256

SHA-2 : 1234

The Digest is: 9f64a747e1b97f131fabb6b447296c9b6f21e79fb3c5356e6c77e89b6a806a

(그림 3-5)는 전자서명 알고리즘인 RSASSA-PKCS1-v1_5를 사용하여 전자서명을 생성하고 검증하는 예제이다.

☑ (그림 3-5) RSASSA-PKCS1-v1_5를 사용한 전자서명/검증 예제

```
var public_key_object = null;
var tobeSigned = convertStringToArrayBufferView("RSA Signature");

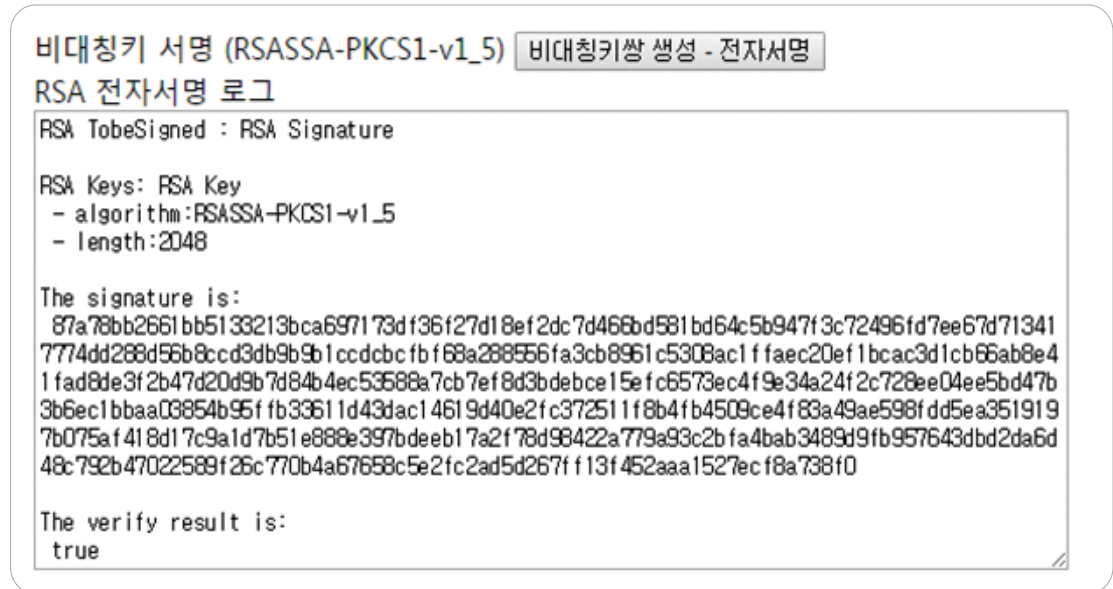
// 키쌍 생성 알고리즘(RSASSA-PKCS1-v1_5) 과 인자 설정
var algorithmKeyGen = {
  name: "RSASSA-PKCS1-v1_5", //사용 알고리즘 설정
  modulusLength: 2048, //인자(키 길이, e) 설정
  publicExponent: new Uint8Array([0x01, 0x00, 0x01]),
  hash: {
    name: "SHA-256"
  }
};

var algorithmSign = {
  name: "RSASSA-PKCS1-v1_5" //서명에 사용할 알고리즘 설정
};

//키 쌍 생성에 성공하면, 해달키로 전자서명
window.crypto.subtle.generateKey(algorithmKeyGen, false, ["sign","verify"]).then(
function(key) {
  public_key_object = key.publicKey;
  // 전자서명 생성
  var sig = window.crypto.subtle.sign(algorithmSign, key.privateKey, tobeSigned);
  return sig;
}),
console.error.bind(console, "Unable to generate a key")
).then(
function (sig)
{
  console.log("The signature is: "+ ab2Hex(sig));
  var verify = window.crypto.subtle.verify(algorithmSign, public_key_object, sig,tobeSigned);
  return verify;
}
).then(
function (verify)
{
  console.log("The verify result is: "+ verify);
}
);
```

(그림 3-6)은 (그림 3-5)의 예제를 사용하여 전자서명 및 검증을 수행한 화면이다.

☑ (그림 3-6) RSA 전자서명 및 검증 수행 화면



(그림 3-7)은 암호화 알고리즘(RSA-OAEP)을 사용하여 비대칭키 기반의 암호화/복호화를 수행하는 예제이다.

☑ (그림 3-7) RSA-OAEP를 사용한 암호화/복호화 예제

```
var private_key_object = null;
var ivData = window.crypto.getRandomValues(new Uint8Array(16)); //암호화에 사용할 iv 설정

// 키쌍 생성 알고리즘(RSA-OAEP) 과 인자 설정
var algorithmKeyGen = {
  name: "RSA-OAEP", //사용 알고리즘 설정
  modulusLength: 2048, //인자(키 길이, e) 설정
  publicExponent: new Uint8Array([0x01, 0x00, 0x01]),
  hash: {
    name: "SHA-256"
  }
};

var algorithmEnc = {
  name: "RSA-OAEP", //사용할 알고리즘 설정,
  iv: ivData //iv 설정
};

//키 쌍 생성에 성공하면, 해당키로 암호문 생성
window.crypto.subtle.generateKey(algorithmKeyGen, false, ["encrypt","decrypt"]).then(
function(key) {
  private_key_object = key.privateKey;
  var tobeEncrypted = convertStringToArrayBufferView("RSA Encrypt");
  var enc = window.crypto.subtle.encrypt(algorithmEnc, key.publicKey, tobeEncrypted);
  return enc;
}),
console.error.bind(console, "Unable to generate a key")
).then(
function (enc) //생성된 암호문 표시
{
  console.log("The cipher data is: "+ ab2Hex(enc));
  var dec = window.crypto.subtle.decrypt(algorithmEnc, private_key_object, enc);
  return dec;
}
).then(
function(dec) // 복호화된 원문 표시
{
  console.log("The decrypted data is: "+ ab2Hex(dec));
}
);
```

(그림 3-8)은 (그림 3-7)의 예제를 사용하여 비대칭키 기반의 암호화/복호화를 수행한 화면이다.

☑ (그림 3-8) 비대칭키 암호화/복호화 예제

비대칭키 암호 복호(RSA OAEP) 비대칭키쌍 생성 - 암호/복호

RSA 전자서명 로그

RSA (OAEP) TobeEncrypted : RSA Encrypt

RSA Keys: RSA Key

- algorithm:RSA-OAEP
- length:2048

The encrypt is:

```
591ef0e0439d15ca1fef5ad51beb462ce2537631fb1bd1ee3598c2d886aa0924e2270acb3b54d904f5421
c416cfla7b3a493648228d0ac305fc30971c5853cbae7b85a54b3bffea384504fb95e799a670dac0e416be
a1ebee4ba54debfc229c741ffa4e2ad204afffe9b852ecbd85defce585f85e91faeb885612e2a86b840a2
dfb1cc173c59ea56d046935b9281de57636281978cd6105f29b43514985243d62a6a79cde0f92588657e62
7491ee5dda78ba3c24b31cbbd26a6122fa2f4ed7e404fb1c8721d7745a113e395aa182cbee9e41d6a35e53
9c23a6482dc79f6fdaaa83285ac5983ba0ff3f67209ea34c42535472
```

The decrypted data is:

RSA Encrypt



제4장 HTML5 기반 국산 블록암호 알고리즘 사용 방법

제1절 국산 블록암호 알고리즘 소개(SEED, HIGHT)

제2절 암호 알고리즘 운영모드 및 패딩 방법

제3절 HTML5기반 국산 블록암호 알고리즘 사용 방법

04

제4장 HTML5 기반 국산 블록암호 알고리즘 사용 방법

본 장에서는 다음과 같이 ISO/IEC 18033 국제 표준에 등록된 국산 블록암호 알고리즘인 SEED, HIGHT의 소개, 암호 알고리즘 운영 모드 및 패딩 방법, HTML5 기반 국산 블록암호 알고리즘 사용 방법을 설명한다.

제1절. 국산 블록암호 알고리즘 소개(SEED, HIGHT)

SEED는 전자상거래, 금융, 무선통신 등에서 전송되는 개인정보와 같은 중요한 정보를 보호하기 위해 1999년 2월 한국인터넷진흥원과 국내 암호전문가들이 순수 국내기술로 개발한 128비트 블록암호 알고리즘이다. 1999년에는 128비트 키를 지원하는 SEED 128(128/128)을 개발하였으며, 2009년 암호 알고리즘 활용성 강화를 위해 256비트 키를 지원하는 SEED 256(128/256)을 개발하였다.

또한, HIGHT(HIGH security and light weight) 암호 알고리즘은 RFID, USN 등과 같이 저전력·경량화를 요구하는 컴퓨팅 환경에서 기밀성을 제공하기 위해 2005년 KISA, ETRI 부설연구소 및 고려대가 공동으로 개발한 64비트(64/128) 블록암호 알고리즘이다.

제2절 암호 알고리즘 운영모드 및 패딩 방법

1. 운영모드

운영모드란, 블록암호의 블록 길이보다 긴 길이의 메시지를 암호화하기 위한 방법에 대한 정의이다. 이러한 운영모드는 블록암호와는 독립적으로 정의된다. 대표적으로 가장 널리 이용되는 블록암호 운영모드에는 ECB(Electronic Code Book) 모드, CBC(Cipher Block Chaining) 모드, CFB(CipherText FeedBack)모드, OFB(Output FeedBack) 모드, CTR(CounTeR) 모드가 있으며, 본 문서에서는 KISA에서 구현하여 배포하는 ECB, CBC, CTR 운영모드에 대해서만 설명한다.

• ECB 운영모드

ECB(Electronic Code Book) 모드는 평문 블록 자체를 그대로 암호화하는 운영모드로서 동일한 내용을 갖는 평문블록은 이에 대응되는 동일한 암호문 블록으로 암호화하는 운영모드이다.

ECB 모드의 암호화 과정은 평문 블록(Pi)을 입력 블록 (ii)으로 설정하고, 이를 암호화한 출력 블록(Oi)을 암호문 블록(Ci)으로 설정한다.

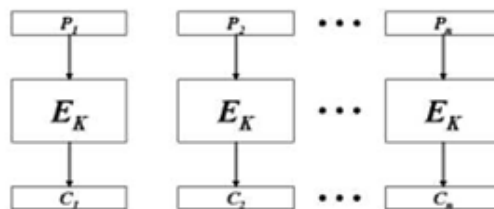
☑ (그림 4-1) ECB 암호화 과정

```

o 입력 :  $P=\{P_1, \dots, P_n\}$ ,  $K$ 
o 출력 :  $C=\{C_1, \dots, C_n\}$ 
o 처리과정
  for  $i=1$  to  $n$ 
     $I_i = P_i$ 
     $O_i = E_K(I_i)$ 
     $C_i = O_i$ 

```

☑ (그림 4-2) ECB 암호화



ECB 모드의 복호화 과정은 암호문 블록(Ci)을 입력 블록(Ii)으로 설정하고, 이를 복호화한 출력 블록(Oi)을 평문 블록(Pi)으로 설정한다.

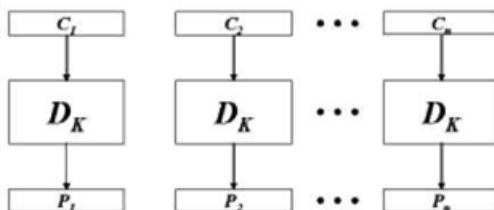
☑ (그림 4-3) ECB 복호화 과정

```

o 입력 :  $C=\{C_1, \dots, C_n\}$ ,  $K$ 
o 출력 :  $P=\{P_1, \dots, P_n\}$ 
o 처리과정
  for  $i=1$  to  $n$ 
     $I_i = C_i$ 
     $O_i = D_K(I_i)$ 
     $P_i = O_i$ 

```

☑ (그림 4-4) ECB 복호화



• CBC 운영모드

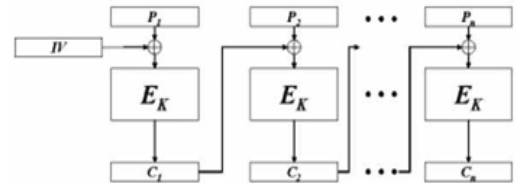
CBC(Cipher Block Chaining)모드는 전 단계의 암호·복호화 결과가 현 단계에 영향을 주는 운영모드이다.

CBC 모드의 암호화 과정은 현 단계에서 평문 블록(Pi)과 전 단계의 암호문 블록 (Ci-1)을 벡터적 논리합 연산한 결과를 현 단계의 입력 블록 (Ii)으로 설정하고, 이를 암호화한 출력 블록(Oi)을 현 단계의 암호문 블록(Ci)으로 설정한다.

✓ (그림 4-5) CBC 암호화 과정

o 입력 : $P=\{P_1, \dots, P_n\}$, K , IV
o 출력 : $C=\{C_1, \dots, C_n\}$
o 처리과정
 $C_0 = IV$
for $i=1$ to n
 $I_i = P_i \oplus C_{i-1}$
 $O_i = E_K(I_i)$
 $C_i = O_i$

✓ (그림 4-6) CBC 암호화

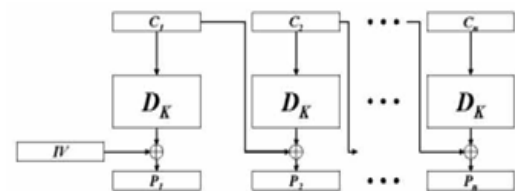


CBC 모드의 복호화 과정은 현 단계의 암호문 블록(Ci)을 입력 블록(Ii)으로 설정하고, 이를 복호화한 출력 블록(Oi)을 전 단계의 입력 블록(Ii-1)인 암호문 블록(Ci-1)과 베타적 논리합 연산한 결과를 현 단계의 평문 블록(Pi)으로 한다.

✓ (그림 4-7) CBC 복호화 과정

o 입력 : $C=\{C_1, \dots, C_n\}$, K , IV
o 출력 : $P=\{P_1, \dots, P_n\}$
o 처리과정
 $C_0 = IV$
for $i=1$ to n
 $I_i = C_i$
 $O_i = D_K(I_i)$
 $P_i = C_{i-1} \oplus O_i$

✓ (그림 4-8) CBC 복호화



• CTR 운영모드

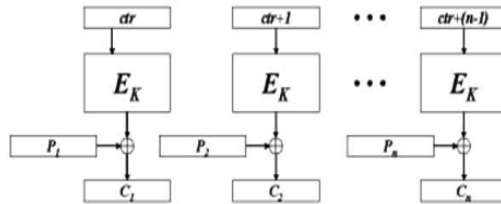
CTR(CounTeR) 모드는 각 단계에 따라 증가되는 카운트 블록을 입력 블록으로 사용하는 운영모드이다.

CTR 모드의 암호화 과정은 카운터(ctr+(i-1))를 현 단계의 입력 블록(Ii)으로 하여 암호화한 출력 블록(Oi)을 평문 블록(Pi)과 베타적 논리합 연산을 수행함으로 암호문 블록(Ci)을 생성한다.

☑ (그림 4-9) CTR 암호화 과정

o 입력 : $P=\{P_1, \dots, P_n\}$, K , ctr
o 출력 : $C=\{C_1, \dots, C_n\}$
o 처리과정
for $i=1$ to n
 $I_i = ctr + (i-1) \bmod 2^{128}$
 $O_i = E_K(I_i)$
 $C_i = P_i \oplus O_i$

☑ (그림 4-10) CTR 암호화

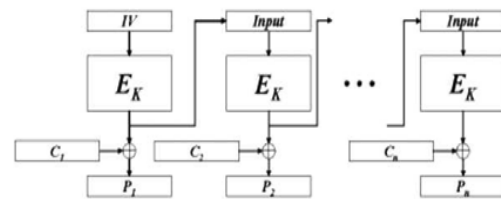


CTR 모드의 복호화 과정은 카운터 ($ctr+(i-1)$)를 현 단계의 입력 블록(I_i)으로 하여 암호화한 출력 블록(O_i)을 암호문 블록(C_i)과 베타적 논리합 연산을 수행함으로 평문 블록(P_i)을 생성한다.

☑ (그림 4-11) CTR 복호화 과정

o 입력 : $C=\{C_1, \dots, C_n\}$, K , ctr
o 출력 : $P=\{P_1, \dots, P_n\}$
o 처리과정
for $i=1$ to n
 $I_i = ctr + (i-1) \bmod 2^{128}$
 $O_i = E_K(I_i)$
 $P_i = C_i \oplus O_i$

☑ (그림 4-12) CTR 복호화



2. 패딩 방법

특정 길이의 평문을 SEED 128비트 암호 알고리즘으로 암호화하기 위해 여러 개의 128비트 블록으로 나눌 때, 마지막 블록을 정확히 128비트 블록의 크기로 맞추는 것은 쉽지 않다. 예를 들어, 300비트의 메시지를 128비트의 블록으로 나눌 경우, $300 = 128 + 128 + 44$ 로 나뉘어 세 개의 블록을 구성하게 되는 데, 이때 마지막 블록이 44비트로 128비트를 만족하지 못한다. 이 경우 메시지의 마지막 블록이 128비트가 되도록 나머지 부족한 84비트를 채워주어야 한다. 이렇게 부족한 부분을 채워 블록의 배수로 맞추는 것을 패딩이라고 하며, 복호화 시 평문의 길이를 복구할 수 있도록 패딩 방법을 고려해야 한다.

ECB, CBC 모드는 평문 블록을 암호화의 입력으로 사용하기 때문에, 평문 데이터의 크기가 128비트의 양의 정수배가 되어야 하며, 반드시 패딩 방법이 적용되어야만 한다. 그러나, CFB-s,

OFB, CTR 모드의 경우, 마지막 암호화 단계의 평문 블록이 128비트(CFB-s 모드의 경우 s비트)를 만족하지 못하고 m비트가 남아있을 때, 마지막 출력 블록(On) 128비트 중 상위 m비트 (MSBs(On))와 마지막 평문 블록 m비트와 베타적 논리합 연산함으로 아래와 같은 패딩 방법을 적용하지 않을 수 있다.

본 패딩 방법은 ISO / IEC 국제표준 및 PKCS에서 사용되는 방법으로, 적용되는 시스템에 맞도록 사용함을 권고하나, '패딩 방법 1'은 복호화된 평문 데이터의 크기가 명확하지 않은 경우가 발생하기 때문에 평문 데이터의 크기가 명확히 알려져 있는 경우에 사용함을 권고한다.

• 패딩 방법 1

평문 데이터의 크기가 128비트 양의 정수배가 아닐 때, 마지막 평문 블록이 128비트가 되도록 바이트 '00'을 덧붙인다.



입력 블록(48비트) : 4F 52 49 54 48 4D

패딩 블록(128비트) : 4F 52 49 54 48 4D 00 00 00 00 00 00 00 00 00 00

입력 블록(128비트) : 53 45 45 44 41 4C 47 A8 3E D1 80 F1 29 DC 4A 78

패딩 블록(128비트) : 53 45 45 44 41 4C 47 A8 3E D1 80 F1 29 DC 4A 78

• 패딩 방법 2

평문 데이터의 크기가 128비트의 양의 정수배가 아닐 경우, 마지막 평문 블록이 128비트가 되도록 바이트 '80' 덧붙인 후 나머지에 모두 '00'바이트를 덧붙이고, 평문 블록의 크기가 128비트의 양의 정수배일 경우, 추가적인 128비트 '80 · · · 00' 블록을 추가한다.)



입력 블록(48비트) : 4F 52 49 54 48 4D

패딩 블록(128비트) : 4F 52 49 54 48 4D 80 00 00 00 00 00 00 00 00 00

입력 블록(128비트) : 53 45 45 44 41 4C 47 A8 3E D1 80 F1 29 DC 4A 78

패딩 블록(256비트) : 53 45 45 44 41 4C 47 A8 3E D1 80 F1 29 DC 4A 78
80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

• 패딩 방법 3

평문 데이터의 크기가 128비트의 양의 정수배가 아닐 경우, 마지막 평문 블록이 128비트가 되도록 덧붙이기가 필요한 바이트 수 'xx · · · xx'를 덧붙이고, 평문 블록의 크기가 128비트의 양의 정수배일 경우, 추가적인 128비트 '10 · · · 10' 블록을 추가한다.



입력 블록(48비트) : 4F 52 49 54 48 4D

패딩 블록(128비트) : 4F 52 49 54 48 4D 0A 0A 0A 0A 0A 0A 0A 0A 0A

입력 블록(128비트) : 53 45 45 44 41 4C 47 A8 3E D1 AF 07 4A 73 12 2C

패딩 블록(256비트) : 53 45 45 44 41 4C 47 A8 3E D1 AF 07 4A 73 12 2C
10 10 10 10 10 10 10 10 10 10 10 10 10 10 10



제3절 HTML5 기반 국산 블록암호 알고리즘 사용 방법

본 절에서는 국산 블록암호 알고리즘인 SEED(ECB, CBC, CTR)와 HIGHT(ECB, CBC, CTR)에 대해서 각각 3가지 패딩 방식(각 패딩 방법의 상세한 내용은 2절에서 설명)적용하여 자바스크립트로 구현한 내용을 소개한다.

본 안내서에 설명된 알고리즘은 자바스크립트로 구현되었으며(KISA_SEED, KISA_HIGHT), 화면 표시를 위해서 HTML(index.html)을 사용하였다. 기능 확인을 위해서는 브라우저를 사용하여 index.html을 열어 원하는 기능을 선택하면 된다. 실제 웹에서 해당 기능을 사용하고자 하는 경우 웹서버에 해당 파일을 저장하고 접근하면 된다. 웹서버의 설정 등은 본 안내서에서 다루지 않는다.

1. SEED

- SEED 함수 설명

```
kisaSEED.createCipher(Alg, Mode, Key, bEnc, padding);
```

암호화/복호화에 사용될 객체 생성

매개변수:

Alg : 암호화/복호화에 사용할 알고리즘 ("SEED")

Mode : 사용할 운영모드 (ECB, CBC, CTR)

Key : 암호화/복호화에 사용할 키

bEnc : 암호화 시 true, 복호화 시 false

padding: 패딩 방식

반환 값:

암호화/복호화 연산에 사용될 객체

```
cipher.init(initdata);
```

암호화/복호화에 사용되는 초기 값 설정

매개변수:

initdata: 운영모드에 따라 각기 다른 값을 설정

- ECB: 해당 매개변수 사용하지 않음

- CBC: IV 설정

- CTR: Counter 설정

`cipher.update(data);`

암호화/복호화에 사용되는 데이터 설정

매개변수:

data: 암호화/복호화에 사용되는 데이터 설정
암호화 시 평문, 복호화 시 암호문 사용

`cipher.final();`

최종 암호문 또는 복호화된 평문을 반환하는 함수

반환값: 암호화 시 암호문, 복호화 시 평문 사용

다음은 SEED의 각 운영모드에 대한 암호화/복호화 수행코드와 결과화면이다.

1.1 SEED (ECB)



(그림 4-13) SEED-ECB 암호화/복호화 수행코드

```
// 키 설정
var userKey = util.createBuffer(new Uint8Array([
    0x88, 0xE3, 0x4F, 0x8F, 0x08, 0x17, 0x79, 0xF1,
    0xE9, 0xF3, 0x94, 0x37, 0x0A, 0xD4, 0x05, 0x89
]));

// 서명 원문 설정
var data = util.createBuffer(new Uint8Array([
    0xD7, 0x6D, 0x0D, 0x18, 0x32, 0x7E, 0xC5, 0x62,
    0xB1, 0x5E, 0x6B, 0xC3, 0x65, 0xAC, 0x0C, 0x0F
]));

// 암호화 수행
var cipher = kisaSEED.createCipher('SEED', 'ECB', userKey, true, padding);

cipher.init();
cipher.update(data);
var cipherText = cipher.final();

// 복호화 수행
userKey.read = 0;
cipher = kisaSEED.createCipher('SEED', 'ECB', userKey, false, padding);

cipher.init();
cipher.update(cipherText);
var plainText = cipher.final();
```

✓ (그림 4-14) SEED-ECB 암호화/복호화 수행 예제(패딩 방법 1)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: SEED, mode: ECB padding 1

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

cipher text

0f 4e 7f c7 8c 48 d5 ad 95 10 ba d8 98 7b a5 22 6f 8c e6 1a 35 6d 47 a4 08 6d ed 87 39 81 03 cd

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

✓ (그림 4-15) SEED-ECB 암호화/복호화 수행 예제(패딩 방법 2)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: SEED, mode: ECB padding 2

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

cipher text

0f 4e 7f c7 8c 48 d5 ad 95 10 ba d8 98 7b a5 22 d0 18 38 89 00 e9 15 fa 04 0f c4 db f5 44 3d 5a

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

✓ (그림 4-16) SEED-ECB 암호화/복호화 수행 예제(패딩 방법 3)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: SEED, mode: ECB padding 3

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

cipher text

0f 4e 7f c7 8c 48 d5 ad 95 10 ba d8 98 7b a5 22 99 09 af 8b b8 17 43 42 07 6c 86 53 e9 e5 cf b8

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

1.2 SEED (CBC)

✓ (그림 4-17) SEED-CBC 암호화/복호화 수행코드

```
//키 설정
var userKey = util.createBuffer(new Uint8Array([
    0x88, 0xE3, 0x4F, 0x8F, 0x08, 0x17, 0x79, 0xF1,
    0xE9, 0xF3, 0x94, 0x37, 0x0A, 0xD4, 0x05, 0x89
]));

// iv 설정
var iv = util.createBuffer(new Uint8Array([
    0x26, 0x8D, 0x66, 0xA7, 0x35, 0xA8, 0x1A, 0x81,
    0x6F, 0xBA, 0xD9, 0xFA, 0x36, 0x16, 0x25, 0x01
]));

// 서명 데이터 설정
var data = util.createBuffer(new Uint8Array([
    0xD7, 0x6D, 0x0D, 0x18, 0x32, 0x7E, 0xC5, 0x62,
    0xB1, 0x5E, 0x6B, 0xC3, 0x65, 0xAC, 0x0C, 0x0F
]));

// 암호화 수행
var cipher = kisaSEED.createCipher('SEED', 'CBC', userKey, true, padding);

cipher.init(iv);
cipher.update(data);
var cipherText = cipher.final();

// 복호화 수행
userKey.read = 0;
cipher = kisaSEED.createCipher('SEED', 'CBC', userKey, false, padding);

iv.read = 0;
cipher.init(iv);
cipher.update(cipherText);
var plainText = cipher.final();
```

✓ (그림 4-18) SEED-CBC 암호화/복호화 수행 예제(패딩 방법 1)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: SEED, mode: CBC padding 1

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

26 8d 66 a7 35 a8 1a 81 6f ba d9 fa 36 16 25 01

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

cipher text

a2 93 ea e9 d9 ae bf ac 37 ba 71 4b d7 74 e4 27 1c 2c b3 b5 4c ef df b8 79 dd 65 76 6a 2b 06 a7

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

✓ (그림 4-19) SEED-CBC 암호화/복호화 수행 예제(패딩 방법 2)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: SEED, mode: CBC padding 2

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

26 8d 66 a7 35 a8 1a 81 6f ba d9 fa 36 16 25 01

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

cipher text

a2 93 ea e9 d9 ae bf ac 37 ba 71 4b d7 74 e4 27 5c c8 54 b6 15 d7 78 e1 00 03 01 6c 91 09 bf 1d

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

✓ (그림 4-20) SEED-CBC 암호화/복호화 수행 예제(패딩 방법 3)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: SEED, mode: CBC padding 3

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

26 8d 66 a7 35 a8 1a 81 6f ba d9 fa 36 16 25 01

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

cipher text

a2 93 ea e9 d9 ae bf ac 37 ba 71 4b d7 74 e4 27 4e 5c d7 b3 97 15 50 26 32 46 6c ab 29
ca ff 18

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

1.3 SEED (CTR)

✓ (그림 4-21) SEED-CTR 암호화/복호화 수행코드

```
// 키 설정
var userKey = util.createBuffer(new Uint8Array([
    0x88, 0xE3, 0x4F, 0x8F, 0x08, 0x17, 0x79, 0xF1,
    0xE9, 0xF3, 0x94, 0x37, 0x0A, 0xD4, 0x05, 0x89
]));

// CTR 설정
var counter = util.createBuffer(new Uint8Array([
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe
]));

// 서명 데이터 설정
var data = util.createBuffer(new Uint8Array([
    0xD7, 0x6D, 0x0D, 0x18, 0x32, 0x7E, 0xC5, 0x62,
    0xB1, 0x5E, 0x6B, 0xC3, 0x65, 0xAC, 0x0C, 0x0F
]));

// 암호화 수행
var cipher = kisaSEED.createCipher('SEED', 'CTR', userKey, true, padding);

cipher.init(counter);
cipher.update(data);
var cipherText = cipher.final();

// 복호화 수행
userKey.read = 0;
cipher = kisaSEED.createCipher('SEED', 'CTR', userKey, false, padding);

counter.read = 0;
cipher.init(counter);
cipher.update(cipherText);
var plainText = cipher.final();
```

✓ (그림 4-22) SEED-CTR 암호화/복호화 수행 예제

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: SEED, mode: CTR

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 fe

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

cipher text

54 1e 1c c4 57 a6 08 3e e9 fb 8a 9c 32 27 41 ed

plain text

d7 6d 0d 18 32 7e c5 62 b1 5e 6b c3 65 ac 0c 0f

2. HIGHT

• HIGHT 함수 설명

```
kisaHIGHT.createCipher(Alg, Mode, Key, bEnc, padding);
```

암호화/복호화에 사용될 객체 생성

매개변수:

Alg : 암호화/복호화에 사용할 알고리즘 ("HIGHT")

Mode : 사용할 운영모드 (ECB, CBC, CTR)

Key : 암호화/복호화에 사용할 키

bEnc : 암호화 시 true, 복호화 시 false

padding: 패딩 방식

반환 값:

암호화/복호화 연산에 사용될 객체

```
cipher.init(initdata);
```

암호화/복호화에 사용되는 초기 값 설정

매개변수:

initdata: 운영모드에 따라 각기 다른 값을 설정

– ECB: 해당 매개변수 사용하지 않음

– CBC: IV 설정

– CTR: Counter 설정

```
cipher.update(data);
```

암호화/복호화에 사용되는 데이터 설정

매개변수:

data: 암호화/복호화에 사용되는 데이터 설정

암호화 시 평문, 복호화 시 암호문 사용

```
cipher.final();
```

최종 암호문 또는 복호화된 평문을 반환하는 함수

반환값: 암호화 시 암호문, 복호화 시 평문 사용

다음은 HIGHT의 각 운영모드에 대한 암호화/복호화 수행코드와 결과화면이다.

2.1 HIGHT (ECB)



(그림 4-23) HIGHT-ECB 암호화/복호화 수행코드

```
//키 설정
var userKey = new Uint8Array([
    0x88, 0xE3, 0x4F, 0x8F, 0x08, 0x17, 0x79, 0xF1,
    0xE9, 0xF3, 0x94, 0x37, 0x0A, 0xD4, 0x05, 0x89
]);

// 서명 데이터 설정
var data = util.createBuffer(new Uint8Array([
    0xD7, 0x6D, 0x0D, 0x18, 0x32, 0x7E, 0xC5, 0x62
]));

var cipher = kisaHIGHT.createCipher('HIGHT', 'ECB', userKey, true, padding);

cipher.init();
cipher.update(data);
var cipherText = cipher.final();

// 복호화 수행
cipher = kisaHIGHT.createCipher('HIGHT', 'ECB', userKey, false, padding);

cipher.init();
cipher.update(cipherText);
var plainText = cipher.final();
```

☑ (그림 4-24) HIGHT-ECB 암호화/복호화 수행 예제(패딩 방법 1)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: HIGHT, mode: ECB padding 1

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

plain text

d7 6d 0d 18 32 7e c5 62

cipher text

e4 bc 2e 31 22 77 e4 dd 74 25 8e 03 ad 89 6d 79

plain text

d7 6d 0d 18 32 7e c5 62

✓ (그림 4-25) HIGHT-ECB 암호화/복호화 수행 예제(패딩 방법 2)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: HIGHT, mode: ECB padding 2

user key

88 e3 4f 8f 06 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

plain text

d7 6d 0d 18 32 7e c5 62

cipher text

e4 bc 2e 31 22 77 e4 dd c0 64 1c 5e 9c 7d d7 f6

plain text

d7 6d 0d 18 32 7e c5 62

✓ (그림 4-26) HIGHT-ECB 암호화/복호화 수행 예제(패딩 방법 3)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: HIGHT, mode: ECB padding 3

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

plain text

d7 6d 0d 18 32 7e c5 62

cipher text

e4 bc 2e 31 22 77 e4 dd 80 f2 12 8d 11 ac 4e 9a

plain text

d7 6d 0d 18 32 7e c5 62

2.2 HIGHT (CBC)

☑ (그림 4-27) HIGHT-CBC 암호화/복호화 수행코드

```
//키 설정
var userKey = new Uint8Array([
    0x88, 0xE3, 0x4F, 0x8F, 0x08, 0x17, 0x79, 0xF1,
    0xE9, 0xF3, 0x94, 0x37, 0x0A, 0xD4, 0x05, 0x89
]);

// iv 설정
var iv = util.createBuffer(new Uint8Array([
    0x26, 0x8d, 0x66, 0xa7, 0x35, 0xa8, 0x1a, 0x81
]));

// 서명 데이터 설정
var data = util.createBuffer(new Uint8Array([
    0xD7, 0x6D, 0x0D, 0x18, 0x32, 0x7E, 0xC5, 0x62
]));

// 암호화 수행
var cipher = kisaHIGHT.createCipher('HIGHT', 'CBC', userKey, true, padding);

cipher.init(iv);
cipher.update(data);
var cipherText = cipher.final();

// 복호화 수행
cipher = kisaHIGHT.createCipher('HIGHT', 'CBC', userKey, false, padding);

iv.read = 0;
cipher.init(iv);
cipher.update(cipherText);
var plainText = cipher.final();
```

☑ (그림 4-28) HIGHT-CBC 암호화/복호화 수행 예제(패딩 방법 1)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: HIGHT, mode: CBC padding 1

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

26 8d 66 a7 35 a8 1a 81

plain text

d7 6d 0d 18 32 7e c5 62

cipher text

9c 8f a0 a5 9f 9e 36 31 99 ae f9 88 b5 7c 2b 2a

plain text

d7 6d 0d 18 32 7e c5 62

✓ (그림 4-29) HIGHT-CBC 암호화/복호화 수행 예제(패딩 방법 2)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: HIGHT, mode: CBC padding 2

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

26 8d 66 a7 35 a8 1a 81

plain text

d7 6d 0d 18 32 7e c5 62

cipher text

9c 8f a0 a5 9f 9e 36 31 1e 48 8c ec 88 78 1c e4

plain text

d7 6d 0d 18 32 7e c5 62

☑ (그림 4-30) HIGHT-CBC 암호화/복호화 수행 예제(패딩 방법 3)

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: HIGHT, mode: CBC padding 3

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

26 8d 66 a7 35 a8 1a 81

plain text

d7 6d 0d 18 32 7e c5 62

cipher text

9c 8f a0 a5 9f 9e 36 31 1d c2 2a 4a 46 61 a3 df

plain text

d7 6d 0d 18 32 7e c5 62

2.3 HIGHT (CTR)

☑ (그림 4-31) HIGHT-CTR 암호화/복호화 수행코드

```
//키 설정
var userKey = new Uint8Array([
    0x88, 0xE3, 0x4F, 0x8F, 0x08, 0x17, 0x79, 0xF1,
    0xE9, 0xF3, 0x94, 0x37, 0x0A, 0xD4, 0x05, 0x89
]);

//카운터 설정
var counter = util.createBuffer(new Uint8Array([
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xfe
]));

// 서명 데이터 설정
var data = util.createBuffer(new Uint8Array([
    0xD7, 0x6D, 0x0D, 0x18, 0x32, 0x7E, 0xC5, 0x62
]));

// 암호화 수행
var cipher = kisaHIGHT.createCipher('HIGHT', 'CTR', userKey, true, padding);

cipher.init(counter);
cipher.update(data);
var cipherText = cipher.final();

// 복호화 수행
cipher = kisaHIGHT.createCipher('HIGHT', 'CTR', userKey, false, padding);

counter.read = 0;
cipher.init(counter);
cipher.update(cipherText);
var plainText = cipher.final();
```

☑ (그림 4-32) HIGHT-CTR 암호화/복호화 수행 예제

PADDING 1	PADDING 2	PADDING 3
HIGHT ECB	HIGHT CBC	HIGHT CTR
SEED ECB	SEED CBC	SEED CTR

알고리즘 정보

algorithm: HIGHT, mode: CTR

user key

88 e3 4f 8f 08 17 79 f1 e9 f3 94 37 0a d4 05 89

iv(or CTR)

00 00 00 00 00 00 00 fe

plain text

d7 6d 0d 18 32 7e c5 62

cipher text

64 bd f0 e7 f4 da 58 a5

plain text

d7 6d 0d 18 32 7e c5 62





제5장 구현 유의사항

제1절 기밀 정보의 저장

제2절 웹서비스 보호

제5장 구현 유의사항

HTML5 기반의 암호 기술은 기존 별도의 확장 프로그램(ActiveX, 플러그인)이 사용자의 운영 체제에서 동작하는 것에 비해 사용자 에이전트에서 동작하는 차이가 있다. 이에 따라 기존의 운영체제의 보안 취약점 이외에 브라우저상에서의 보안 취약점에 대해서도 추가적인 검토가 필요하다.

제1절 기밀 정보의 저장

모든 암호 기술은 키를 포함한 각종 기밀 정보가 안전하게 저장된다는 가정 하에 데이터를 암호화하고, 복호화하는 동작을 수행한다. 기존 확장 프로그램의 경우 기밀 정보는 메모리와 실행코드에 존재하게 되어 해당 정보를 추출하기에는 많은 노력이 필요했다. 하지만, HTML5 방식의 암호 기술은 사용자 에이전트에서 암호 동작을 수행하고, 기밀 정보 역시 사용자 에이전트에 저장하게 되어, 최근 사용자 에이전트는 서비스가 제공되고 있는 페이지에 대해서 많은 정보를 쉽게 취득할 수 있는 도구를 제공하고 있다.

이에 따라서 저장 방식에 따른 기밀 정보에 대한 보안을 고려하여야 한다.

－ 웹 스토리지

웹 스토리지는 HTML5 표준으로 웹 사이트의 데이터를 웹 브라우저에 저장하는 기존 쿠키의 단점을 개선하기 위해 개발된 저장 방식이다. 웹 스토리지는 동일출처정책(Same-Origin Policy, SOP)에 따라 해당 도메인에서만 접근 가능한 저장소이고, 클라이언트 디바이스에 직접 저장되고 네트워크로 전송되지 않기 때문에 네트워크 레벨에서는 안전할 수 있다. 하지만, 물리적으로 파일로 저장되며, 웹 브라우저 개발자 도구 등으로 쉽게 접근이 가능하며, 수정 및 삭제가 가능하다. 따라서 기밀 정보(암호화 키, 공인인증서 및 개인키 등)를 웹 스토리지에 저장하는 경우 추가적인 암호화, 사용자 인증 등의 보안 수단을 반드시 고려하여야 한다.

－ 인덱싱된 데이터베이스 (IndexedDB)

웹 스토리지의 대용량 문제와 DB 구조 복잡화로 단점을 보완하고자 만든 인덱싱된 데이터베이스는 자바스크립트 객체(Object)를 저장하고 인덱스에 의해 접근하는 방식을 제공한다. 웹 스토리지와 마찬가지로 동일출처정책을 따르며, 클라이언트 디바이스에 직접 저장되고 네트워크로는 전송되지 않지만, 웹 브라우저 개발자 도구 등을 통해 접근이 가능하므로, 추가적인 암호화, 사용자 인증의 보안 수단을 반드시 고려하여야 한다.

자바스크립트의 객체를 저장할 수 있는 인덱싱된 데이터베이스를 WebCrypto API의 저장소로

사용하는 경우 객체의 데이터로의 변환이 필요하지 않고, 설정에 따라 개인키에 대한 직접 접근이 제한될 수 있는 장점이 있다.

따라서, 웹 브라우저 내에서 암호화 키를 저장하고자 하는 경우 웹 스토리지보다는 인덱싱된 데이터베이스를 우선적으로 사용할 것을 권고한다.

– W3C 파일 API

HTML5 표준에서는 웹 브라우저에서 파일 시스템에 직접 접근할 수 있는 파일 API를 표준으로 정의하고 있다. 파일 API를 드래그앤드롭 기능과 같이 사용하는 경우 사용자의 PC에 저장되어 있는 중요 데이터를 보다 편리하게 브라우저에 제공할 수 있다. 국내의 경우 공인인증서를 인증서 이동 포맷인 PKCS#12 형태로 변환하고 이를 웹 브라우저에서 사용하는 서비스가 가능하다.

이 경우 사용자가 공인인증서와 개인키를 사용자 PC에 별도로 관리하게 되며, 이에 따라 복사, 유출, 분실 등에 대한 위험은 여전히 존재하게 된다.

제2절 웹서비스 보호

HTML5로 구현되는 암호 기술은 모든 기능을 웹 표준 언어로 구현하고 웹 어플리케이션을 통해서 서비스되기 때문에 웹 취약점에 대한 대응, 자바스크립트 소스에 대한 보호 등 콘텐츠 보호 기능이 반드시 필요하다.

– 자바스크립트 난독화

자바스크립트는 일반적으로 클라이언트측 언어로 작성되고 사용되기 때문에 소스 코드가 공개되는 단점을 가지고 있다. 소스코드가 공개되는 경우 숨기고자 하는 로직이나 알고리즘이 그대로 공개되어 악의적인 공격이 더욱 쉬워지는 일이 발생한다.

이에 대응하기 위해 프로그램의 일부 또는 전체를 변경하고 이를 통해 코드의 가독성을 낮추어 역공학을 대비하는 난독화(Obfuscation)를 적용할 수 있다. 난독화 기법으로는 변수명 등 문자열을 다른 문자로 대체하여 사람이 해석하게 어렵게 만드는 방식, 소스코드를 잘게 나누어 변수에 저장한 후 실제 사용시 재 조합하는 방식, eval 함수, XOR 등의 인코딩을 이용하는 방식 등이 있으며 필요한 기능에 따라 적절한 방식을 선택하여 사용한다.

자바스크립트 난독화는 해독이 불가능한 기술이 아닌 어렵게 하는 기술이기 때문에 그 자체로 완벽한 보안 방식이 아니라 소스코드 보호를 위한 최소한의 요구 사항이며, 다른 다양한 보호 방안과 함께 사용되어야 한다. 단, 성능을 고려한 난독화물 도입이 필요하다.

– 자바스크립트 암호화

자바스크립트 소스를 보호하기 위해서 소스코드를 암호화하는 방식을 사용할 수 있다. 자바스크립트 소스를 암호화하기 위해서는 웹 브라우저에서 생성한 암호화 키를 서버와 공유하고 이를 통해 자바스크립트 소스를 암호화하는 방식을 사용한다.

하지만, SSL이나 기존의 확장 프로그램 방식과는 달리 자바스크립트는 실제 사용을 위해서는 반드시 복호화가 되어야하고, 복호화를 위한 자바스크립트 코드, 암호화에 사용된 키 등은 개발자 도구를 통해 쉽게 추출될 수 있다. 따라서, 자바스크립트 암호화는 일반적인 암호화와는 달리 통신 구간에서만 보안을 제한적으로 제공한다고 볼 수 있다.

– 자바스크립트 무결성 확인

자바스크립트는 언어의 특징상 모든 소스코드가 노출되는 것 이외에도 DOM 객체, 자바스크립트에 대한 위변조가 가능하다. 특히 자바스크립트의 소스코드에 대한 위변조는 주요 기능에 대한 심각한 보안 문제가 발생하게 되며 이를 방지하기 위해 자바스크립트 실행 시 무결성 검증을 통해 소스코드가 임의로 변경되지 않았음을 반드시 확인해야한다.

무결성 검증은 서버 측 검사(Server-side Integrity Test)와 클라이언트 측 검사(Client-side Integrity Test)로 나뉘며 각각 아래와 같은 특징을 가진다.

- ❖ **서버 측 검사** : 서버에서 콘텐츠 배포 시 해당 콘텐츠에 대한 무결성 정보를 포함하여 배포하고, 클라이언트에서는 특정 시점(다음 페이지 요청 시 또는 주기적으로)에 검증을 요청하는 방식이다. 서버가 최종 검사 주체라는 점에서 보안성이 우수하지만, 검증을 위해서는 별도의 통신이 필요하고, 페이지가 복잡한 경우 콘텐츠 무결성 검사를 위한 정보 생성이 매우 복잡해진다는 단점이 있다.
- ❖ **클라이언트측 검사** : 콘텐츠 배포 시 클라이언트에서 무결성 정보를 생성하고 이를 실시간 또는 주기적으로 검사하는 방식이다. 검증을 위해 별도의 통신이 필요하지 않지만, 무결성 검사를 수행하는 모듈 역시 클라이언트에 있기 때문에 해당 모듈에 대한 보안 수단을 강구해야한다.

1989년 개발된 웹은 발전을 거듭하여 현재 실생활에 필수 불가결한 서비스가 되었다. 도입 초기에 단순히 정보 공유의 목적에서 벗어나 많은 서비스를 제공하게 됨에 따라 많은 확장 프로그램을 사용하게 되었고, 이를 다시 표준에서 수용하기 위해 HTML5 표준이 제정되었다.

국내에서는 인터넷 뱅킹, 전자 결제 서비스 등을 제공하기 위해 다양한 확장 프로그램을 사용하여 암호 기능을 제공하고 있다. HTML5 표준 중 WebCrypto API를 사용하는 경우 암호 기능의 일부를 별도 확장 프로그램 없이 사용할 수 있다.(단, 국내 PKI 환경에서는 VID의 사용, SEED, ARIA, KCDSA 등의 알고리즘 지원 등에서 WebCrypto와 차이가 있음)

하지만, WebCrypto API를 포함한 HTML5 기반 암호 기술은 기존 확장 프로그램과는 다른 보안 취약점을 가지고 있기 때문에 이에 대한 대비가 필요하다.



부 록

부록 1 용어 정리

부록 2 파일 정보

부록 3 참고 자료

부록1

용어 정리

HTML (HyperText Markup Language)

- 웹 페이지를 작성하기 위한 마크업 언어이며, 제목, 단락, 링크 등 본문의 구조적인 문서를 만들 수 있는 방법을 제공

XML (eXtensible Markup Language)

- 특수 목적의 마크업 언어를 만드는 용도에 권장되는 다목적 마크업 언어

XHTML (Extensible Hypertext Markup Language)

- HTML과 동등한 표현 능력을 지닌 마크업 언어로, HTML보다 엄격한 문법을 가지는 XML의 응용이며 XHTML 1.0은 2000년 1월 26일, W3C의 권고안이 됨

HTML5

- 월드 와이드 웹 컨소시엄(W3C)에서 개발 중인 차세대 웹 표준으로, 하이퍼텍스트 생성 언어(HTML)의 다음 버전으로 플러그 인 없이 웹에서 원하는 애플리케이션을 만들 수 있도록 HTML을 발전시킨 것

URL(Uniform Resource Locator)

- 인터넷에서 파일, 뉴스그룹과 같은 각종 자원을 표시하기 위한 표준화된 논리 주소. 사용할 프로토콜 (http, ftp 등), 주 컴퓨터의 이름과 주소, 파일이 있는 디렉터리 위치, 파일 이름으로 구성됨

HTTP(HyperText Transfer Protocol)

- 인터넷의 월드 와이드 웹(WWW) 서버와 웹(WWW) 브라우저가 파일 등의 정보를 송수신하는 데 사용되는 클라이언트/서버 규약. 웹(WWW) 브라우저의 화면상에서 URL(uniform resource locator)를 지정하는데 사용됨

W3C(World Wide Web Consortium)

- 월드 와이드 웹 브라우저/서버 기술의 표준화를 추진하고 있는 교육/연구 기관 및 관련 회사들의 단체

사용자 에이전트 (User Agent)

- 특수 목적의 마크업 언어를 만드는 용도에 권장되는 다목적 마크업 언어

동일출처정책(Same-Origin Policy, SOP)

- 웹 어플리케이션 보안모델의 주요 개념으로, 웹 브라우저는 동일한 출처가 아니면 첫 번째 웹 페이지의 자바스크립트는 두 번째 웹 페이지의 데이터를 접근할 수 없음

문서 객체 모델(Document Object Model, DOM)

- 플랫폼/언어 중립적으로 구조화된 문서를 표현하는 W3C의 표준

TLS (Transport Layer Security)

- 현재 널리 사용되고 있는 SSL(Security Sockets Layer) 대신의 차세대 안전 통신 규약. SSL에 비해 강력한 암호화를 실현할 수 있고 폭넓은 망의 통신 규약에 대응되어 있음

자바스크립트 (Javascript)

- 객체 기반의 스크립트 프로그래밍 언어로 웹 브라우저 내에서 주로 사용하며, 1996년 2월 넷스케이프 네비게이터 2.0에 포함되었음

JSON (Javascript Object Notation)

- 자바 스크립트 객체 표기법. 인터넷에서 자료를 주고받을 때 그 자료를 표현하는 방법으로 자바스크립트의 구문 형식을 따르지만 플랫폼에 독립적인 방식

MSB (Most Significant Bit)

- 비트 단위의 연산에 있어서 그 숫자 값의 크기에 가장 크게 영향을 미치는 유효 숫자이며, 그 숫자를 나타내는 비트 열 중에서 가장 왼쪽의 비트를 의미

암호화 (Encryption, Encipherment)

- 의미를 알 수 없는 형식(암호문)으로 정보를 변환하는 것. 암호문의 형태로 정보를 기억 장치에 저장하거나 통신 회선을 통해 전송함으로써 정보를 보호. 암호화는 암호 키(특정의 비트열)를 사용하여 정보를 암호문으로 변환하는 것

복호화 (Decryption)

- 암호화(Encryption, Encipherment) 과정의 역과정으로 암호 알고리즘에 의하여 암호문을 평문으로 바꾸는 과정

대칭키 암호기술(Symmetric Cryptographic Technique)

- 송·수신자가 동일한 비밀 키를 사용하는 암호기술

비밀키(Secret Key)

- 대칭키 암호기술에서 사용되는 키로 지정된 실체들만이 사용할 수 있음

비대칭 암호기술(Asymmetric Cryptographic Technique)

- 공개키에 의해 정의되는 공개 변환과 개인키에 의해 정의되는 비공개 변환이라는 관련된 두 가지 변환을 사용하는 암호기술. 이 두 변환 사이에는 공개 변환이 알려져 있다 하더라도 비공개 변환을 알아내기가 계산적으로 불가능하다는 속성이 있음

비대칭키 쌍

- 암호화와 복호화에 사용되는 키가 서로 다른 비대칭키 알고리즘에서의 키 쌍

개인키(Private Key)

- 비대칭키 쌍에서 한 실체에게만 사용되는 비밀 키

공개키(Public Key)

- 비대칭키 쌍에서 공개되는 키

서명(전자서명)

- 서명자를 확인하고 서명자가 해당 전자문서에 서명하였음을 나타내는 데 이용하기 위해 해당 전자문서에 첨부되거나 논리적으로 결합된 전자적 형태의 정보

검증(전자서명 검증)

- 이용자의 개인키를 사용하여 생성한 전자서명을 이용자의 공개키를 사용하여 그 내용의 무결함을 확인하는 절차

다이제스트(메시지 다이제스트)

- 임의 길이의 데이터를 해시 함수를 통해 생성된 고정된 길이의 데이터

키 유도

- 마스터키, 패스워드 등의 키에서 랜덤을 사용하여 비밀키를 유도하는 것

비트 유도

- 마스터키, 패스워드 등의 키에서 랜덤을 사용하여 비트를 유도하는 것

키 싸기 (Key Wrap)

- 암호키를 캡슐화(암호화)하도록 고안된 대칭키 암호 알고리즘의 한 분류

키 풀기(Key Unwrap)

- 키 싸기를 통해서 생성된 데이터를 복호화 하는 방법

WebCrypto API

- 사용자 에이전트에 의해 관리되는 암호기능 제공을 위한 저레벨 인터페이스가 정의되어 있는 API로 W3C에 의해 표준화 됨

CryptoKey 객체

- 브라우저에서 WebCrypto API의 기능을 제공하기 제공하는 자바스크립트 객체

부록2

파일 정보

본 안내서에서는 W3C의 WebCrypto API를 이용한 전자서명/검증, 암호화/복호화 기능과 국산 블록암호 알고리즘인 SEED, HIGHT를 이용한 암호화/복호화를 수행할 수 있는 소스코드를 제공한다. 제공되는 소스코드는 KISA_HTML5_CRYPT0.zip으로 제공되며 소스코드의 구성은 아래와 같다.

※ SHA-1 : 676B5379873E2C9C2B50BE78DA546D2D717933EB

index.html

- SEED, HIGHT 블록 암호 알고리즘의 입력/출력을 확인할 수 있는 테스트 페이지

KISA_HIGHT.js

- HIGHT 구현 소스 코드

KISA_SEED.js

- SEED 구현 소스 코드

webcrypto.html

- WebCrypto를 이용한 AES/RSA 알고리즘의 입력/출력을 확인할 수 있는 테스트 페이지

webcrypto.js

- Web Crypto를 이용한 AES/RSA 알고리즘 사용 소스 코드

참고 자료

④ KS(국가표준)

- > KS X ISO IEC 14888 : 부가형 디지털 서명
- > KS X ISO IEC 18033 : 암호 알고리즘

④ TTA(한국정보통신기술협회표준)

- > TTAK,KO-12,0241 : ARIA/SEED 암호알고리즘을 이용한 키 유도함수
- > TTAS,KO-12,0004/R1 : 128비트 블록암호알고리즘(SEED)
- > TTAS,KO-12,0025 : 블록암호알고리즘 SEED의 운영모드
- > TTAS,KO-12,0040/R1 : 64비트 블록암호 HIGHT

④ PKCS(Public-Key Cryptography Standard)

- > RSA 암호 표준, <http://www.ietf.org/rfc/rfc3447.txt>
- > 패스워드 기반 암호 표준, <http://www.ietf.org/rfc/rfc2898.txt>
- > 개인키 정보 문법 표준, <http://www.ietf.org/rfc/rfc5208.txt>

④ KISA

- > 웹 표준 기반 공인인증서비스 도입 및 구현 기술 안내서
- > SEED 128 알고리즘 사양 및 세부명세
- > HIGHT 알고리즘 사양 및 세부 명세
- > 암호기술 구현 안내서
- > 인터넷 이용환경 개선을 위한 기술안내서

④ NIST

- > NIST SP 800-38A : Recommendation for Block Cipher Modes of Operation

④ World Wide Web Consortium

- > Web Cryptography API

안내서 작성 공헌자

본 안내서의 작성을 위해 아래와 같이 여러분들이 공헌하셨습니다.

구분	소속	직위	성명
총괄책임자	한국인터넷진흥원	본부장	조윤희
	한국인터넷진흥원	단장	손경호
참여연구원	한국인터넷진흥원	팀장	박상환
	한국인터넷진흥원	선임연구원	김주혁
	한국인터넷진흥원	선임연구원	김기문
자문위원 및 외부전문가	충남대학교	교수	류재철
	한양대학교	교수	송정환
	국가보안기술연구소	실장	김우환
	에티소프트	실장	강효관
	삼성SDS	수석	선동규
	드림시큐리티	부장	김대식
	펜타시큐리티	부장	강선명
	이니텍	부장	이은배
	케이사인	본부장	배무건

암호이용활성화

HTML5 암호기술 이용 안내서

2015년 12월 인쇄

2015년 12월 발행

발행처 : 미래창조과학부 · 한국인터넷진흥원

경기도 과천시 관문로 47(중앙동, 과천청사 4동)

미래창조과학부

대표전화: 국번없이 1335

서울 송파구 중대로 135(가락동) IT벤처타워

Tel: (02) 405-4118

인쇄처 : 강산

Tel: (02) 2279-8413

(비매품)

■ 본 안내서 내용의 무단 전재를 금하며, 가공 · 인용할 때에는 반드시 미래창조과학부 · 한국인터넷진흥원 HTML5 암호기술 이용 안내서라고 출처를 밝혀야 합니다.